

VisPortfolio – Final Report

Features Implemented	Remain to be Done
• Drag and drop document customization • .pdf and .docx generation • Responsive Screen	• External link integration • Schedule management • Profile and account recovery options

Known Issues

- .docx export's formatting doesn't keep the header formatting from the preview
- Lacking the ability to re-drag an item AFTER adding it to the document

Flyer



What is Visual Portfolio?

A web-based portfolio app for creating, storing, exporting, and organizing important records.

With a document builder, the app saves time, reduces repeat work, and makes it simple to format different documents for different needs.

Features

- build cover letters, resumes, and portfolios
- drag-and-drop user-made items into document builder
- export files as .pdf and .docx
- have records in one place

TEAM

- Aaron Matthews
- Whitney Branch
- Calina Winfield
- Erick Vale

Team

Pixel Improve

Whitney Branch UI/UX & Client Liaison 	Aaron Matthews Code Architecture/Lead Programmer 	Calina Winfield Data Modeler & Documentation Lead 	Erick Vale Project Manager & Testing Lead 
--	---	---	--

Client

Dr. Anca Doloc-Mihu

Looking to get a web app to help faculty keep track of and organize many records related to their academic work. Many of these records are repeated when faculty need to submit required info to different venues. This tool will help faculty enter their data once and then visually select what is needed to automatically generate the requested report (e.g., resume, portfolio). This will allow individuals to log in to the tool and customize the components of their resume/portfolio/records/etc.



Project Abstract

VisPortfolio is a comprehensive web application designed to bridge the gap between static document creation and dynamic asset management. Built with a MEAN (MongoDB, Express, Angular, Node.js) technology stack, VisPortfolio empowers users to build highly customized, interactive forms and documents using an integrated drag-and-drop interface.

The core innovation of VisPortfolio lies in its hybrid content model. Unlike traditional form builders, it features a proprietary Injected Item System that allows users to seamlessly embed pre-made personal or professional assets—such as past project descriptions and category-specific credentials—directly into a live Formeo-based editor.

By transforming the act of "filling out a form" into a "visual composition," VisPortfolio provides an intuitive solution for professionals looking to generate reports, portfolios, and structured documents with a fraction of the manual effort.

Testing Coverage, methods, and results

Coverage targets

1. Backend: 80%+ (tests under testing/*.test.js)
2. Frontend: 70%+ unit coverage (frontend/src/**/*.spec.ts) and E2E coverage for critical flows
3. E2E: full coverage for auth, editing workflow, export

Methods, tooling, and test files

4. Unit (backend): Jest — config jest.config.cjs, run: npm test
5. Files: testing/auth.full.test.js, testing/core.routes.test.js, testing/*.test.js
6. Unit (frontend): Karma + Jasmine — unit_testing_setup.md, run: cd frontend && ng test --watch=false --code-coverage

Files: frontend/src/app/components/dashboard/dashboard.component.spec.ts
frontend/src/app/components/create-item/create-item.component.spec.ts
frontend/src/app/components/document/document.component.spec.ts frontend/src/app/components/form-builder/form-builder.component.spec.ts frontend/src/.../*.spec.ts (other existing spec files)

Integration: Supertest (co-located in testing/)

7. Files: testing/auth.full.test.js, testing/core.routes.test.js
8. E2E: Cypress — EDITING_WORKFLOW_E2E.md and any cypress tests (basic E2E already present)
9. Flows: signup/login, auth, editing workflow, export, SSO (placeholder)
10. Auth flow tests: basic auth covered; SSO tests to add when SSO implemented
11. System / acceptance: system_testing.md for system-level scenarios and acceptance criteria
12. CI: .github/workflows/tests.yml runs root Jest + frontend tests; add coverage enforcement (jest --coverageThreshold, karma coverage thresholds) and make CI fail PRs on unmet criteria

Results & acceptance criteria

13. CI blocks merges if: any test fails OR coverage < target thresholds.
14. Expected pass: all critical auth, dashboard, editing, export flows; no high-severity test failures.
15. Coverage artifacts: generate HTML reports and CI badges; fail build when thresholds not met.
16. SSO handling: mark SSO tests skipped until implementation; require re-run post-SSO release.

Quick run (repo root)

17. npm install
18. npm test
19. cd frontend && npm install
20. ng test --watch=false --code-coverage
21. Run Cypress: npx cypress run

Recommended next tasks (high value)

22. Add SSO E2E and unit tests when SSO implemented (create tests under testing/ and frontend/specs).
 23. Enforce coverage thresholds in CI (jest.config.cjs + karma config) and update .github/workflows/tests.yml to fail PRs.
 24. Add CI step to publish coverage reports and badges; fail if critical E2E flows regress.
 25. Ensure system_testing.md and unit_testing_setup.md are up-to-date and referenced in CI docs.
-

Documentation

How to Run

1. Open terminal and move to backend subfolder: "cd VisPortfolio/backend"
2. run "npm install" to install dependencies
3. Fix audits if some appear with "npm audit fix" (if it still shows some after doing the command, disregard them)
4. In the backend folder, create a .env file (just click on backend and create it there) and add the following:
 - a. MONGO_URL=mongodb+srv://<username>:<Password>@cluster0.btpuj.mongodb.net/VisPortfolio?retryWrites=true&w=majority&appName=Cluster0
 - b. PORT=5001
5. Then go to the terminal, access the backend:
 - a. to access backend: cd backend
6. Afterwards, insert the following commands:
 - a. node server.js
 - b. npm install
 - c. npm audit
 - d. npm audit fix
 - e. npm start
7. After that, go to the MongoDB extension and click on connect, add the following:
 - a. mongodb+srv://<db_username>:<db_password>@cluster0.btpuj.mongodb.net/ - You need to replace the username with the information for user and password from Client
 - b. Example: mongodb+srv://visportfolio:h7j433RFoU79Sk1U@cluster0.btpuj.mongodb.net/

To run the front end

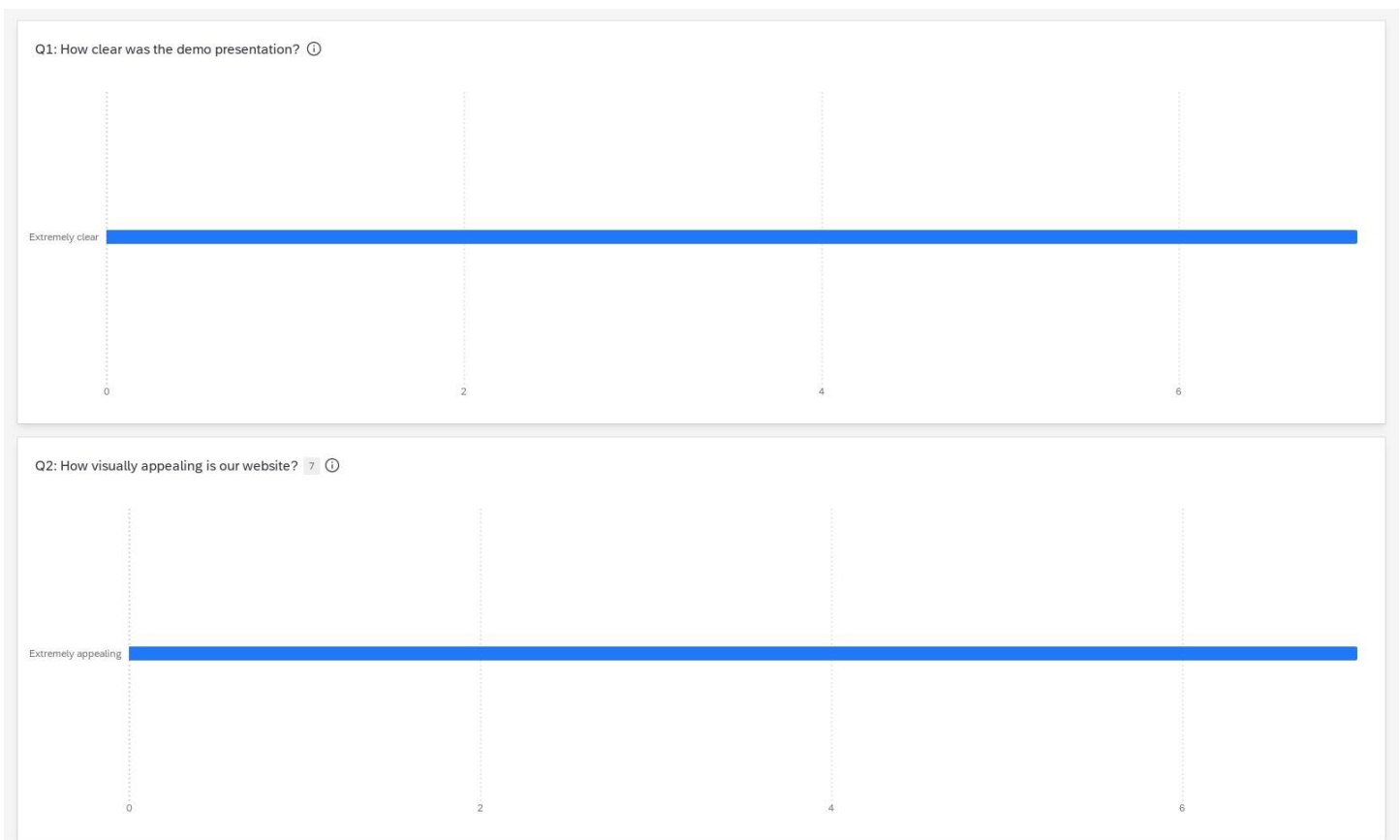
1. Go back to the main directory: cd ..
2. Go to VisPortfolio/frontend on your terminal and run: npm install
3. Next, run with: ng serve

Making an Admin Account

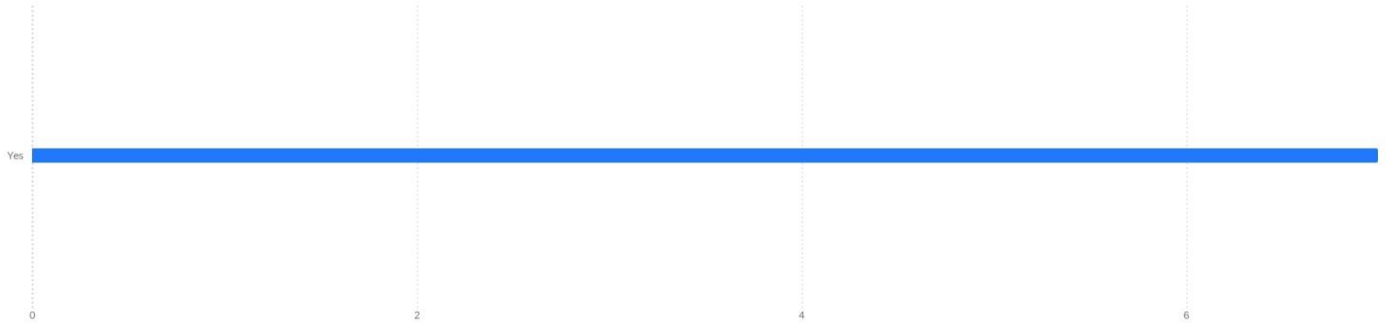
1. Click the 'Get Started' button to sign up for an account on the web app (write down or make sure to remember the email)
2. After getting to the dashboard, logout

3. Open makeAdmin.js file (backend/scripts/makeAdmin.js)
 4. Change line 15 (or a line with "const email = ...") to the email that you want to become an admin
 - a. For example: from [const email = 'test@yahoo.com';] --> to [const email = 'testtest@gmail.com';]
 - b. You will only need to do this once, since an admin will be able to promote/demote any account thereafter to/from admin on the dashboard when logged in
 5. Save the file
 6. In the backend terminal, stop the backend (ctrl + c)
 7. Run the file in the terminal with: node makeAdmin.js
 8. Restart the backend
 9. Log in with the admin account
 10. You SHOULD load into an Admin Dashboard page
 - a. If not, make sure you fully stopped and restarted the backend, and that you put the correct email (from sign up) in the makeAdmin.js file
-

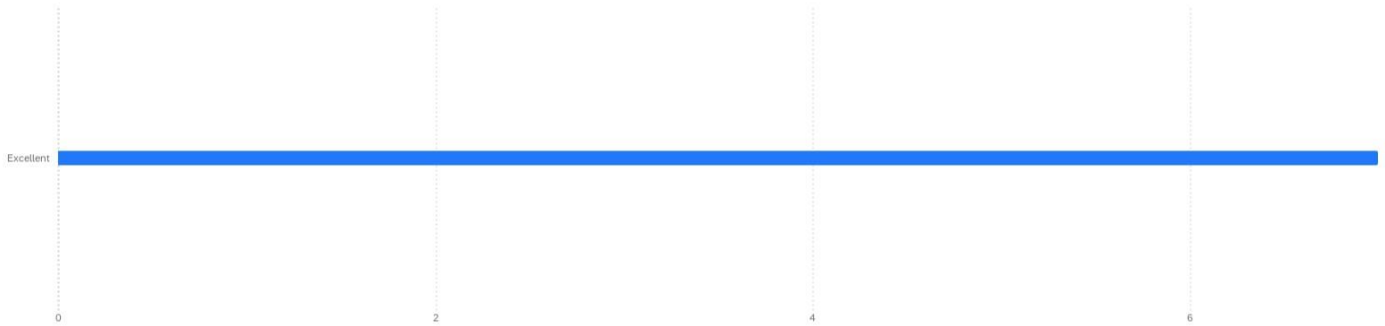
Usability and survey results



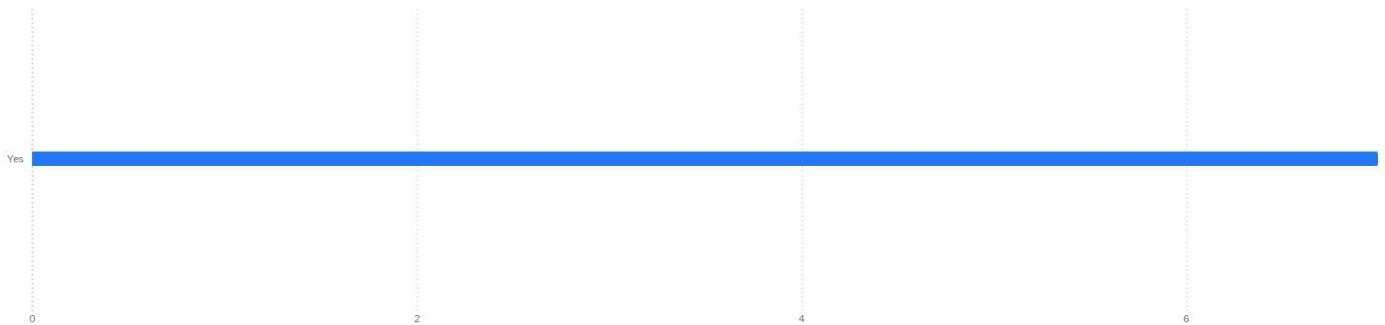
Q3: Did you understand what the app does? 7 ⓘ



Q4: How would you rate the visual design? 7 ⓘ



Q5: Would you personally use this app if available? 7 ⓘ



Q7: What did you like most about the app/demo? 7 ⓘ

What did you like most about the app/demo?

The layout and design of the cards (items, documents)

The admin dashboard and the document builder.

Sleek design

The ease of use

Excellent work -- your demonstration showed your mastery of your work and applying it.

Q8: Do you have any specific feedback for the document builder? 7 ⓘ

Do you have any specific feedback for the document builder?

Having the 'document text blocks' box be smaller, or under the 'your items' card, as to not keep pushing the document builder section down

Being able to move the 'items' after adding them to the document builder's editing area.

great job

Q9: What other overall improvements would you suggest? 7 ⓘ

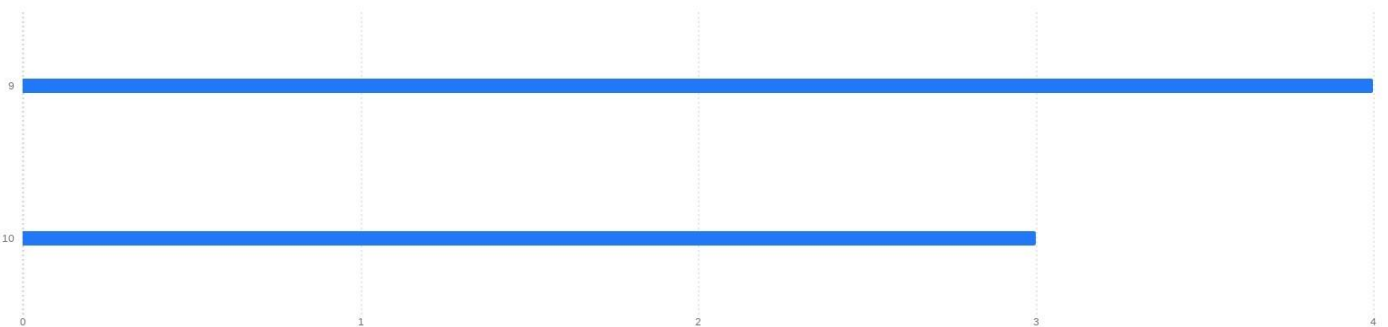
What other overall improvements would you suggest?

None

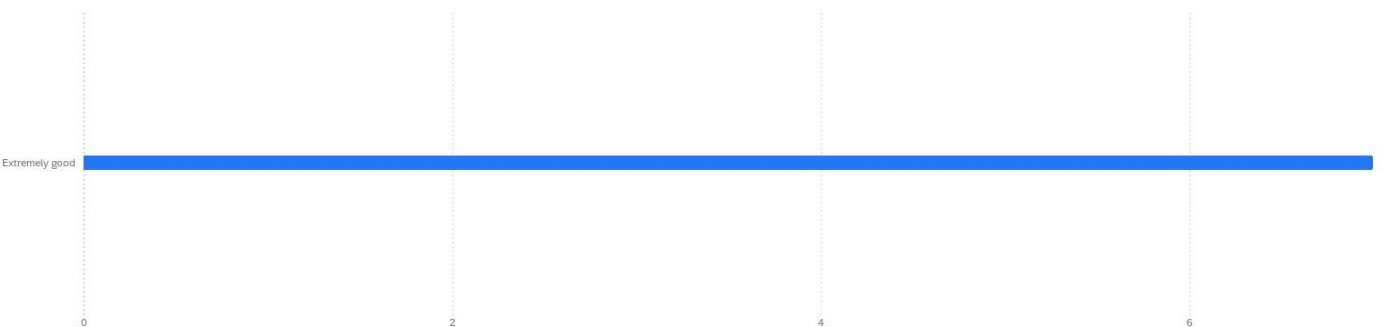
Reducing the floating effect on the 'sign-in' card

none

Q10: How likely is it that you would recommend this portfolio app to a friend or... ⓘ



Q11: Overall, how would you rate the portfolio app? 7 ⓘ



Everyone who took the survey felt that, overall, the web app that we demoed was excellent and was visually appealing. They also understood how to use it and would highly recommend it to a friend or

colleague. For some feedback that was given, specifically for the document builder, someone stated that the 'document text blocks' box should be smaller or moved under the 'your items' card, so as to stop pushing the document builder section down. Another suggestion was to allow the user to be able to move 'items' after they have already been placed in the document builder's editing area. With those being valid and great pieces of feedback, they have been added to the TODO.md for the next team.

Installation

Clone git repo: git clone <https://github.com/GGC-SD/VisPortfolio.git>

Intellectual Property Contract

Project: Visual Portfolio, VisPortfolio

Course: Software Development II

Institution: Georgia Gwinnett College

Date: April 18th, 2026

Purpose and Scope

This document outlines the ownership of Intellectual Property (IP) created during the development of Visual Portfolio. Visual Portfolio is a MEAN stack web application (MongoDB, Express.js, Angular, Node.js) built for Georgia Gwinnett College faculty. It features comprehensive tools for designing, editing, and exporting professional digital portfolios and resumes. Intellectual property includes, but is not limited to, source code, documentation, designs, graphics, and logos produced by the team members in connection with the Visual Portfolio project.

Intellectual Property Terms

- **Ownership:** All team members agree that the project's intellectual property is shared among the stakeholders according to the percentages defined in Section 4.
- **Usage Rights:** Each team member is granted a non-exclusive, perpetual, and royalty-free license to use the project materials for personal portfolios and future non-commercial academic demonstrations.
- **Confidentiality:** Team members agree to protect any sensitive information or trade secrets shared by the client (if applicable) during the development process.

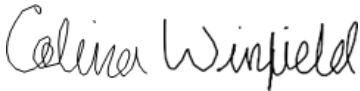
Academic Disclaimer

IMPORTANT: This document is created solely for the fulfillment of course requirements for Software Development II taught through Georgia Gwinnett College. This is a **mock**

agreement and is **not a legally binding contract** in a court of law. It is intended for classroom assessment purposes only.

Stakeholder Ownership & Signatures

The following table identifies all stakeholders, including the Client, Product Owner, and Programming Team Members.

Name	Specific Role(s)	Percent	Signature
Aaron Matthews	Code Architecture	20%	
Whitney Branch	UI/UX Design/Client Liaison	20%	
Calina Winfield	Data Modeler/Document Lead	20%	
Erick Vale	Testing Lead/Project Manager	20%	
Dr. Anca Doloc-Mihu	Client/Product Owner	20%	

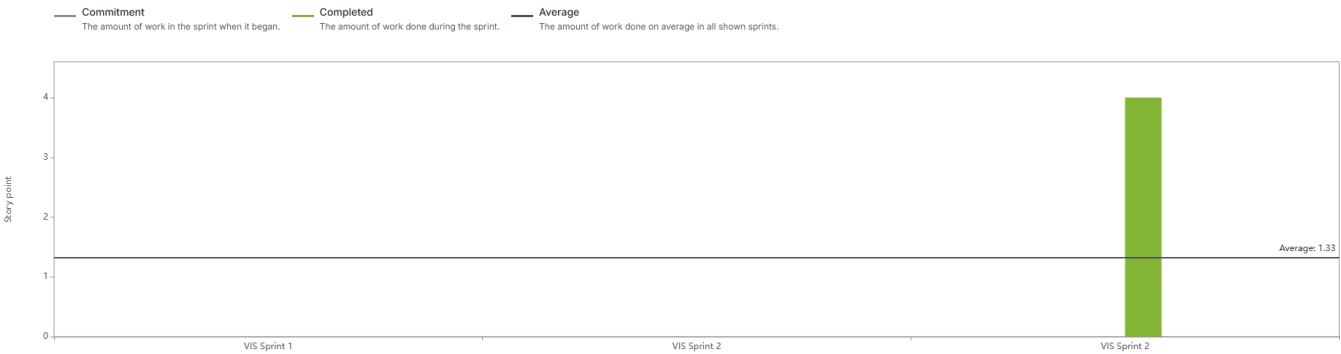
Velocity and Burndown Charts

Final Iteration's Velocity and Burndown Chart

Spaces / Pixel Improve / Reports

Velocity report

> How to read this report



Sprint	Commitment	Completed
VIS Sprint 1	0	0
VIS Sprint 2	0	0
VIS Sprint 2	0	4



Spaces / Pixel Improve / Reports

Burnup report

> How to read this report

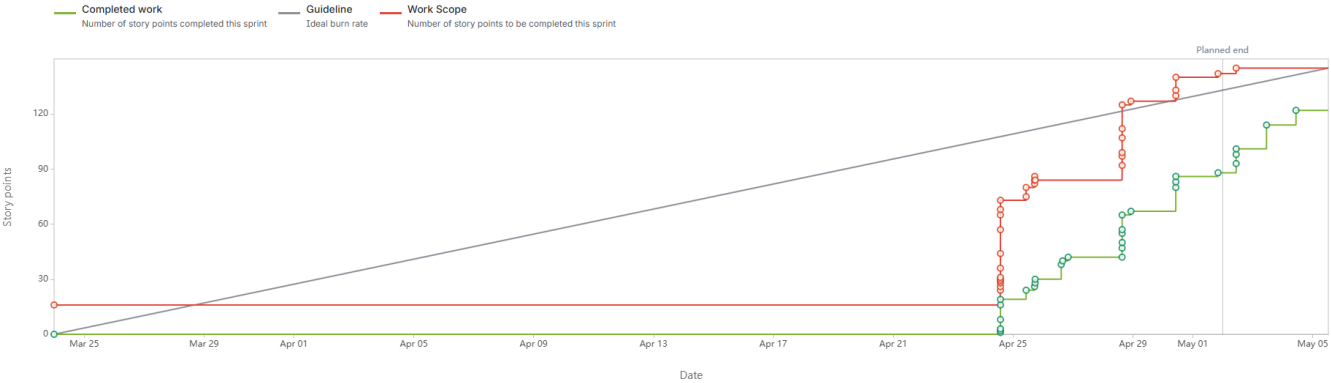
Sprint

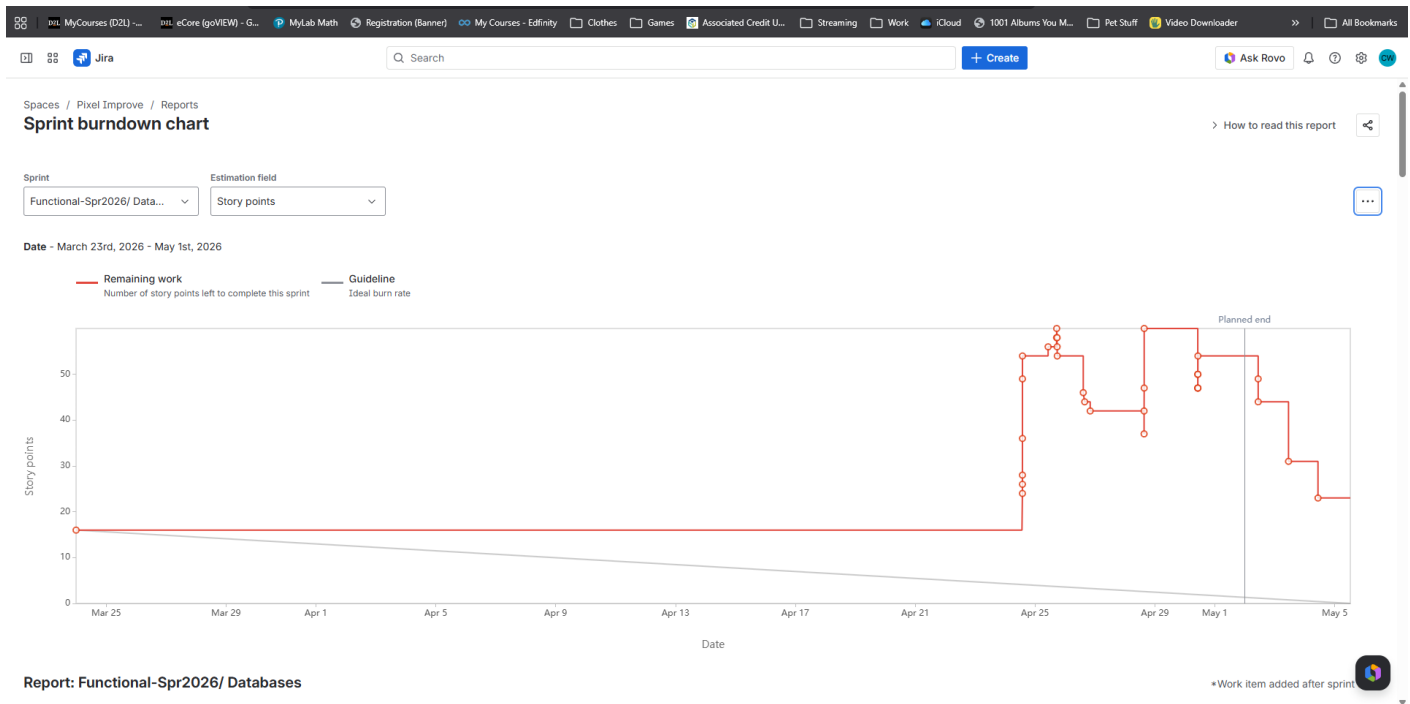
Functional-Spr2026/ Data...

Estimation field

Story points

Date - March 23rd, 2026 - May 1st, 2026





Reflections for ALL Iterations' velocity and burndown charts:

Our velocity charts from the first 2 iterations were a poor display of what we were truly doing behind the scenes. There was a good amount of work being done on the web app that we unfortunately weren't able to display at that time. Same thing with the burndown charts of those iterations; we lacked the full understanding of how to have our backlog/board properly set up to display the progress we were making.

We finally figured out how to properly show progress on our velocity and burndown charts: through having our subtasks be their own tasks, so that their individual progress (time logged, todo vs in-progress vs done) can be seen and tracked.

For the first 2 iterations, we had 6-8 stories with 4-6 subtasks within, which was not good. Because when we made progress on those subtasks or logged time, it doesn't show in the burndown or velocity chart. What's worse is that with us finally figuring out how to do JIRA better, it makes it look like we just recently created all those tasks, which isn't the case.

If this had to be done again, we would immediately put all the subtasks as their own tasks on JIRA, and name them something with "Subtasks 1.2" to indicate that it is a subtask of the larger task 1, for example.